

Compte-rendu

MERN Stack

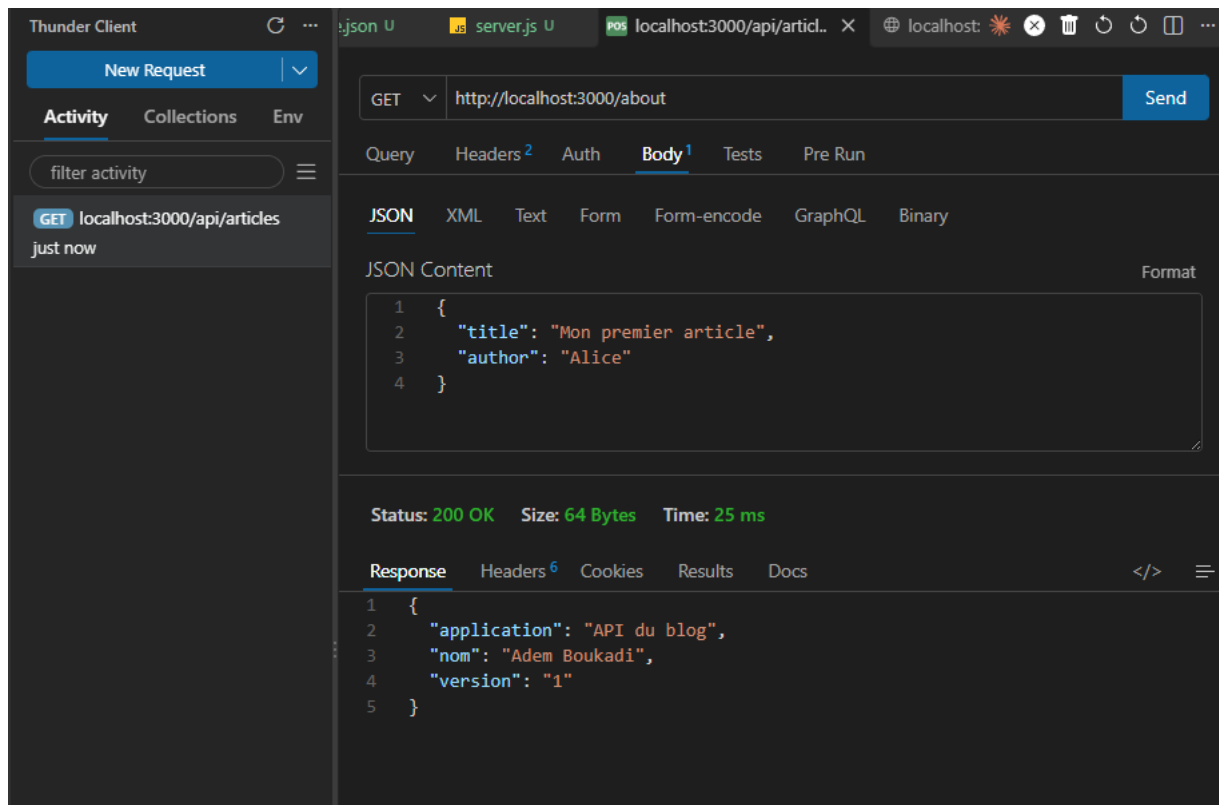
Exercice 1 :

L'étape 1 crée « /about » en ajoute le nom d'application et mon nom avec la version puisque c'est la premier projet je utilise n°1

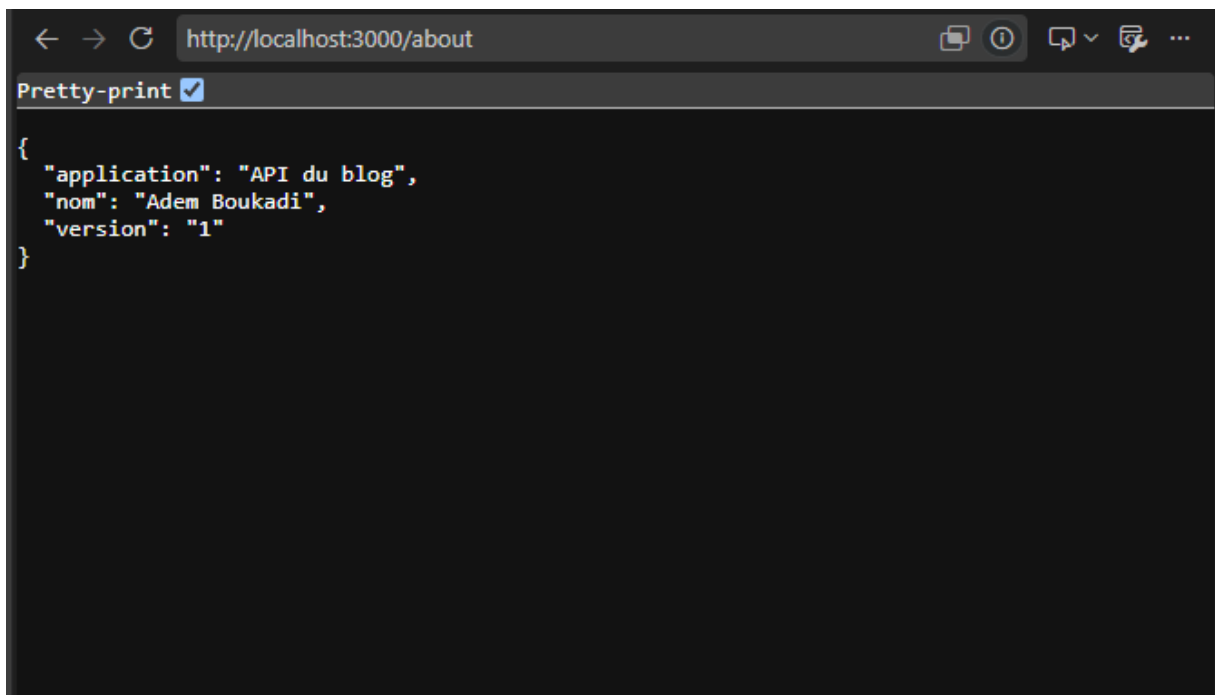
```
app.get("/about", (req, res) => {
  res.json({
    application: "API du blog",
    nom : "Adem Boukadi",
    version: "1",
  });
});

// 4. Démarrer le serveur : il attend les requêtes sur le port 3000
app.listen(PORT, () => {
  console.log(`Serveur disponible sur http://localhost:${PORT}`);
});
```

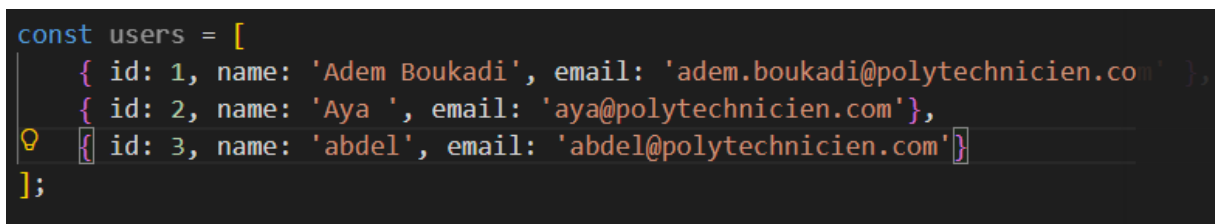
En tester avec postman (mais j'est utilise thunder Client puisque il est dans vscode et plus rapide)



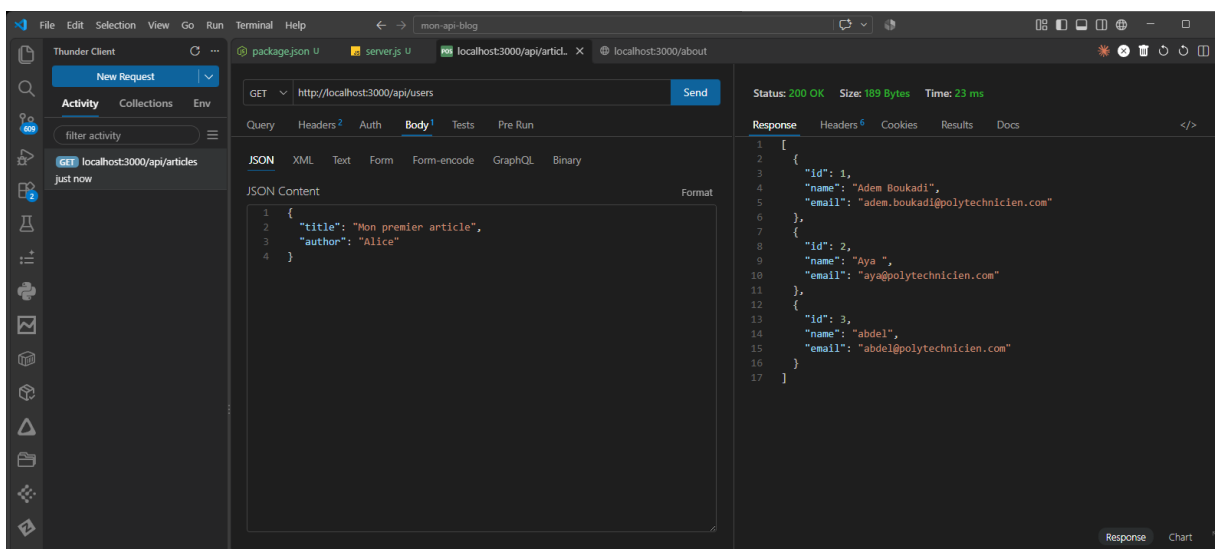
C'est le test web dans le VSCode



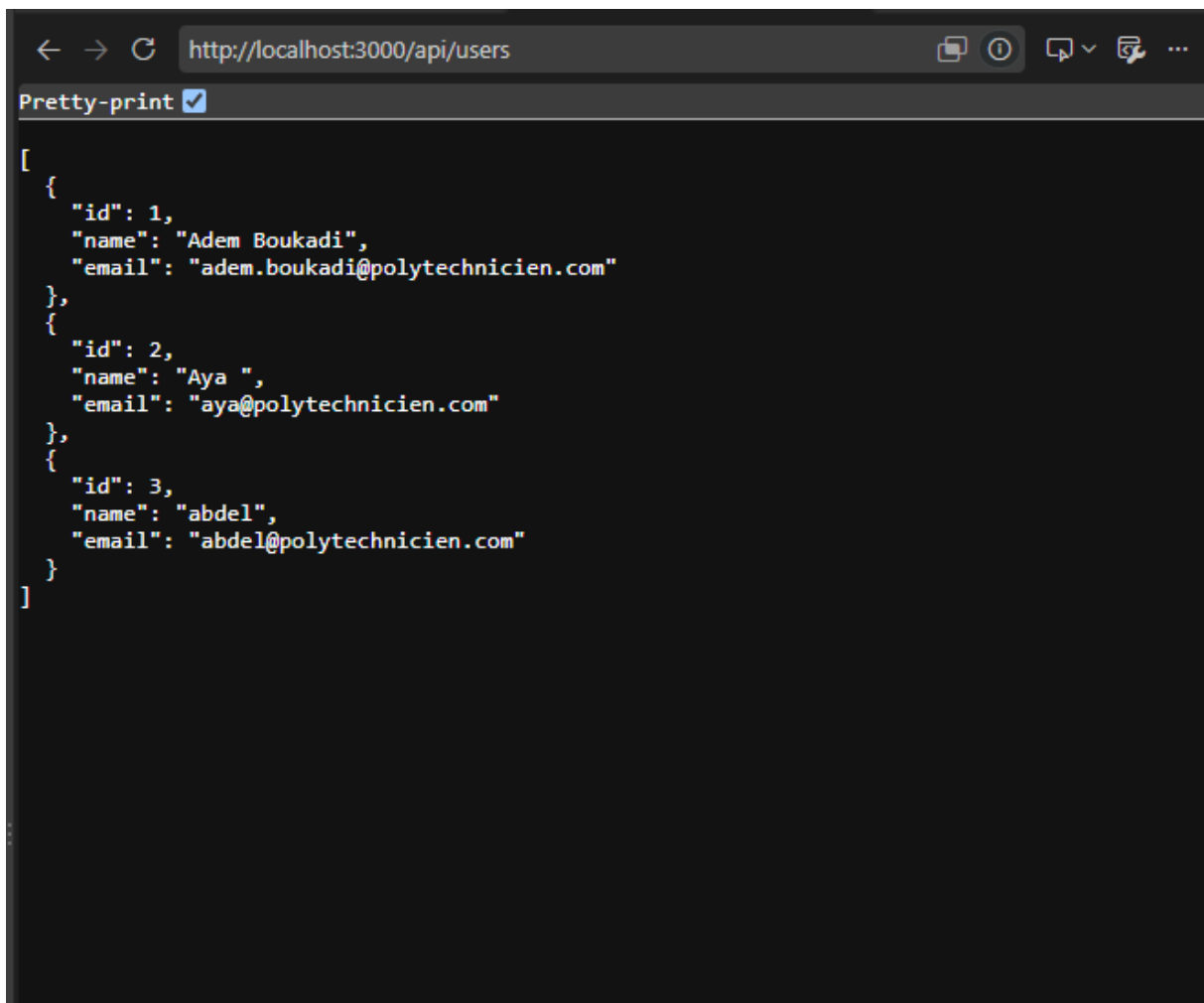
Dans l'étap 2 j'ai creat un const users (j'ajoutte id , nom et email de 3 perssion (la premiere persion c'est moi))



c'est le test avec ThunderClient



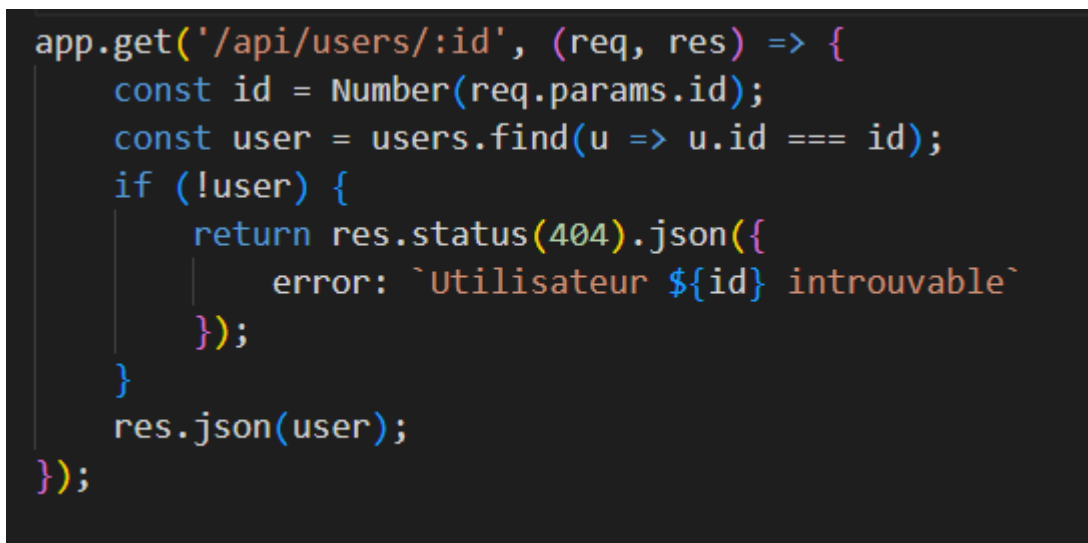
C'est le test avec web



A screenshot of a web browser window. The address bar shows the URL `http://localhost:3000/api/users`. Below the address bar, there is a toggle for 'Pretty-print' which is checked. The main content area displays a JSON array of three user objects, formatted for readability. The first user has an id of 1, name 'Adem Boukadi', and email 'adem.boukadi@polytechnicien.com'. The second user has an id of 2, name 'Aya ', and email 'aya@polytechnicien.com'. The third user has an id of 3, name 'abdel', and email 'abdel@polytechnicien.com'.

```
[
  {
    "id": 1,
    "name": "Adem Boukadi",
    "email": "adem.boukadi@polytechnicien.com"
  },
  {
    "id": 2,
    "name": "Aya ",
    "email": "aya@polytechnicien.com"
  },
  {
    "id": 3,
    "name": "abdel",
    "email": "abdel@polytechnicien.com"
  }
]
```

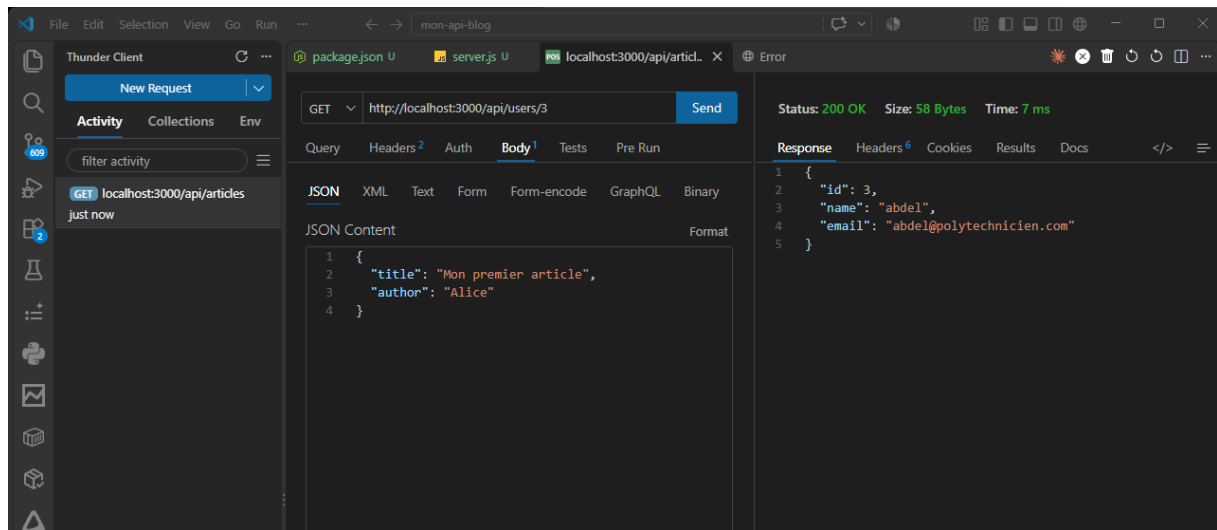
En Etap 3 en ajoute « /api/users » pour les users en create en etap 2



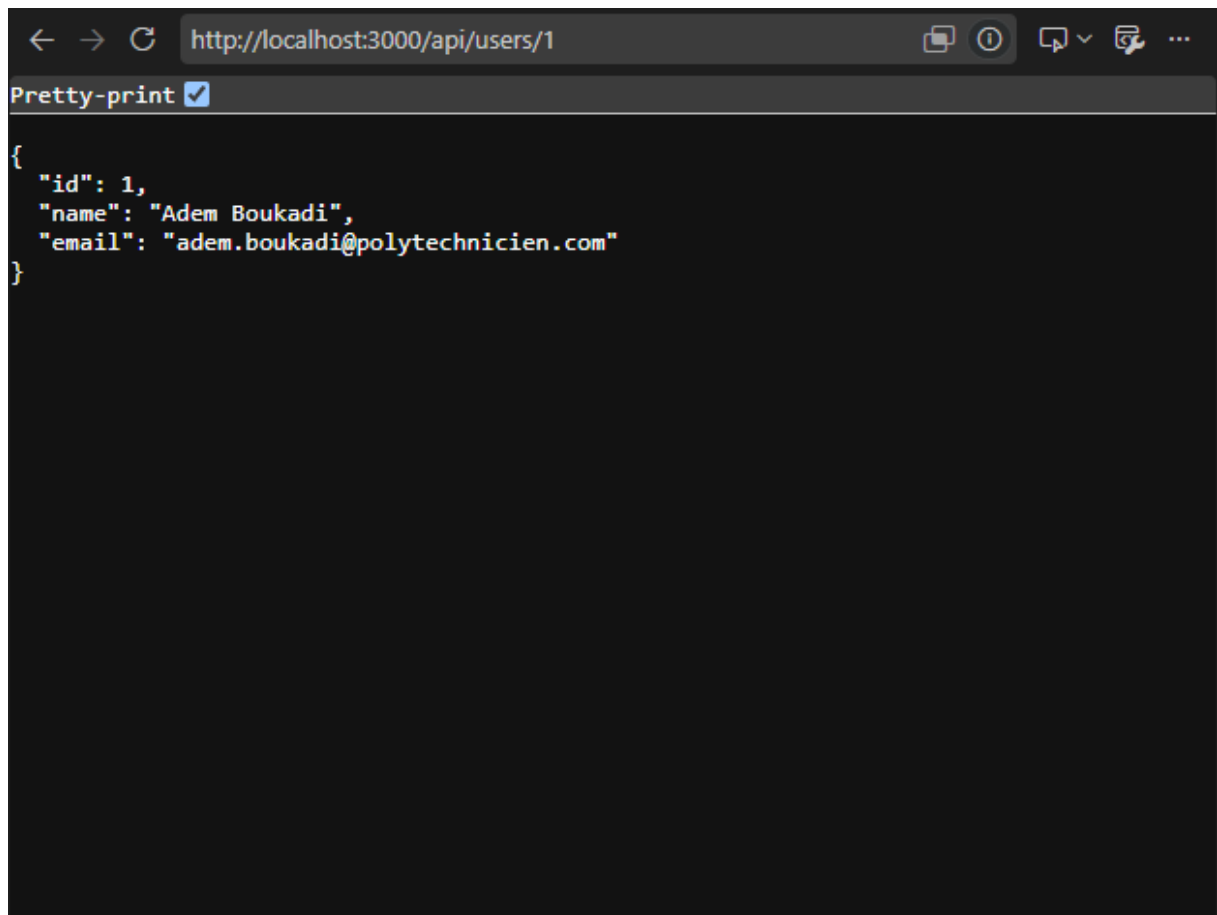
A screenshot of a code editor showing a JavaScript function that serves as a GET route handler. The function takes a request (req) and a response (res) as arguments. It extracts the user ID from the request parameters, finds the user in a 'users' array, and returns a 404 error if the user is not found. Otherwise, it returns the user object as JSON.

```
app.get('/api/users/:id', (req, res) => {
  const id = Number(req.params.id);
  const user = users.find(u => u.id === id);
  if (!user) {
    return res.status(404).json({
      error: `Utilisateur ${id} introuvable`
    });
  }
  res.json(user);
});
```

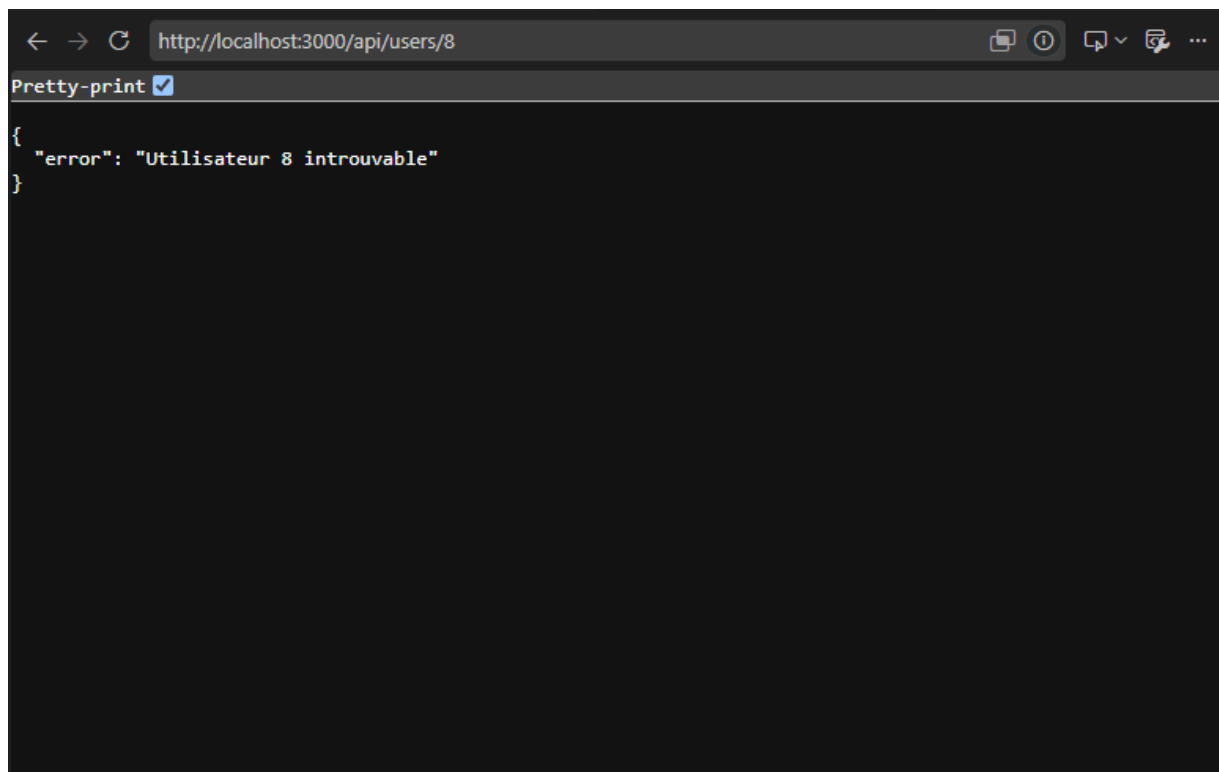
En testé avec Thunder Client



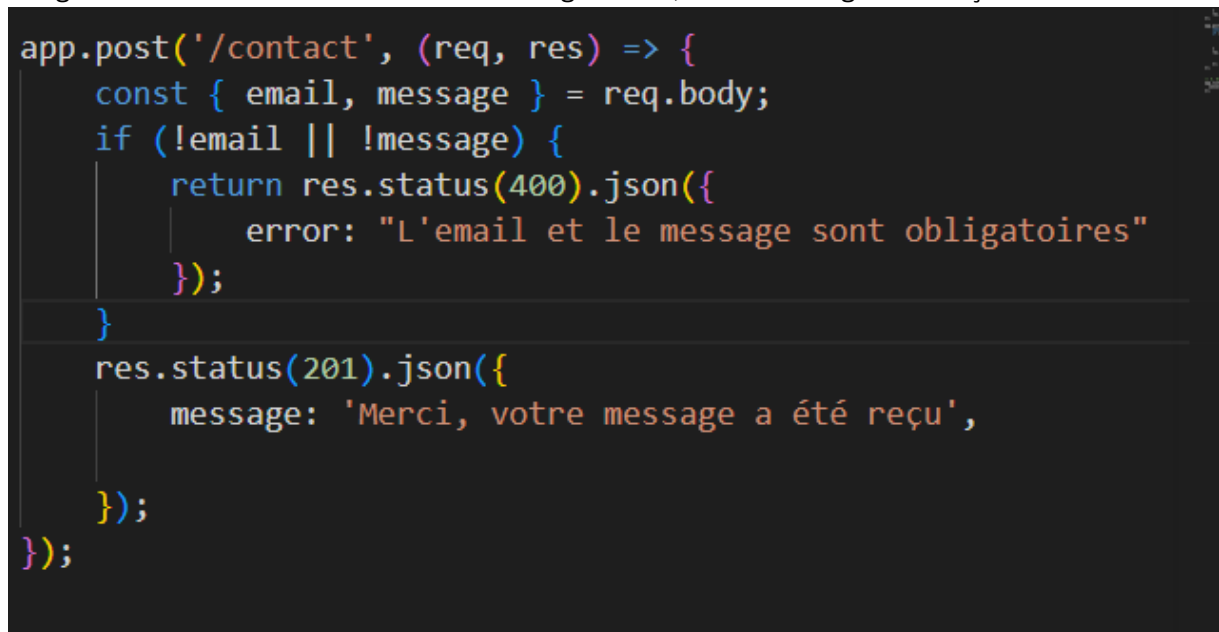
Web test



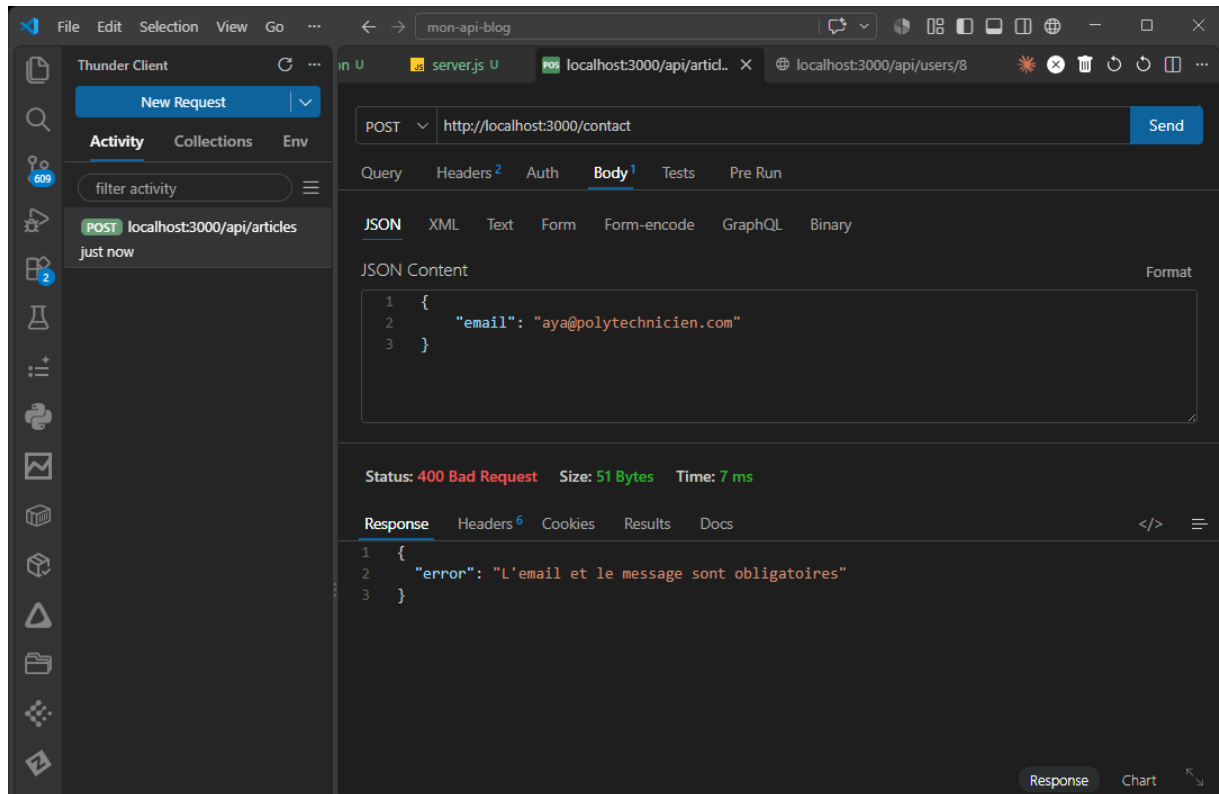
Test l'erreur



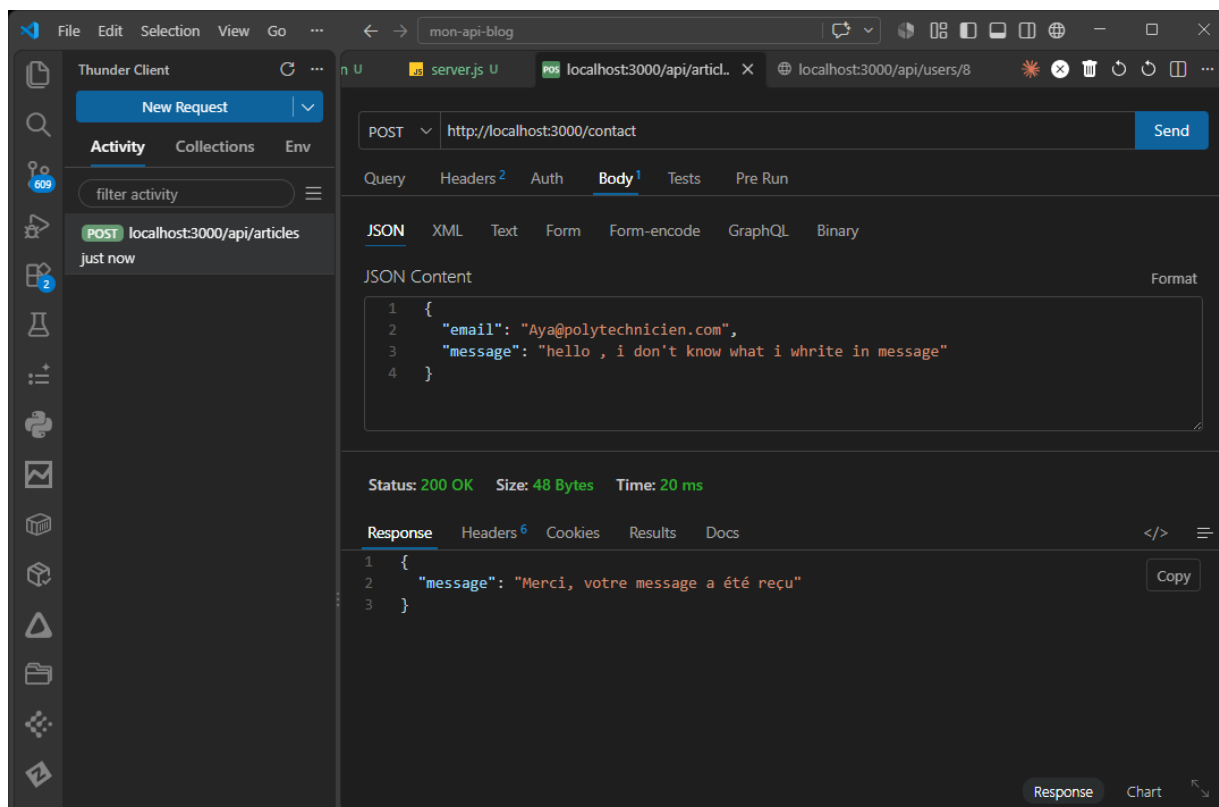
Etap 4 en utilise « /contact » en utilise post en 'email et message' et en utilise « if » is il ya un champ manque ou les deux il affiche le 400 avec un message 'Email et Message sont obligatoires' sinon il affiche 200 avec message 'Merci, votre message a ètè reçu'



C'est le test d'erreur



C'est le test de '200'



L'étap 5 en change le code de app.get avec filtre

```
const users = [
  { id: 1, name: 'Adem Boukadi', email: 'adem.boukadi@polyte
000/contact id: 2, name: 'Aya', email: 'aya@polytechnicien.com'},
  { id: 3, name: 'abdel', email: 'abdel@polytechnicien.com'}
];

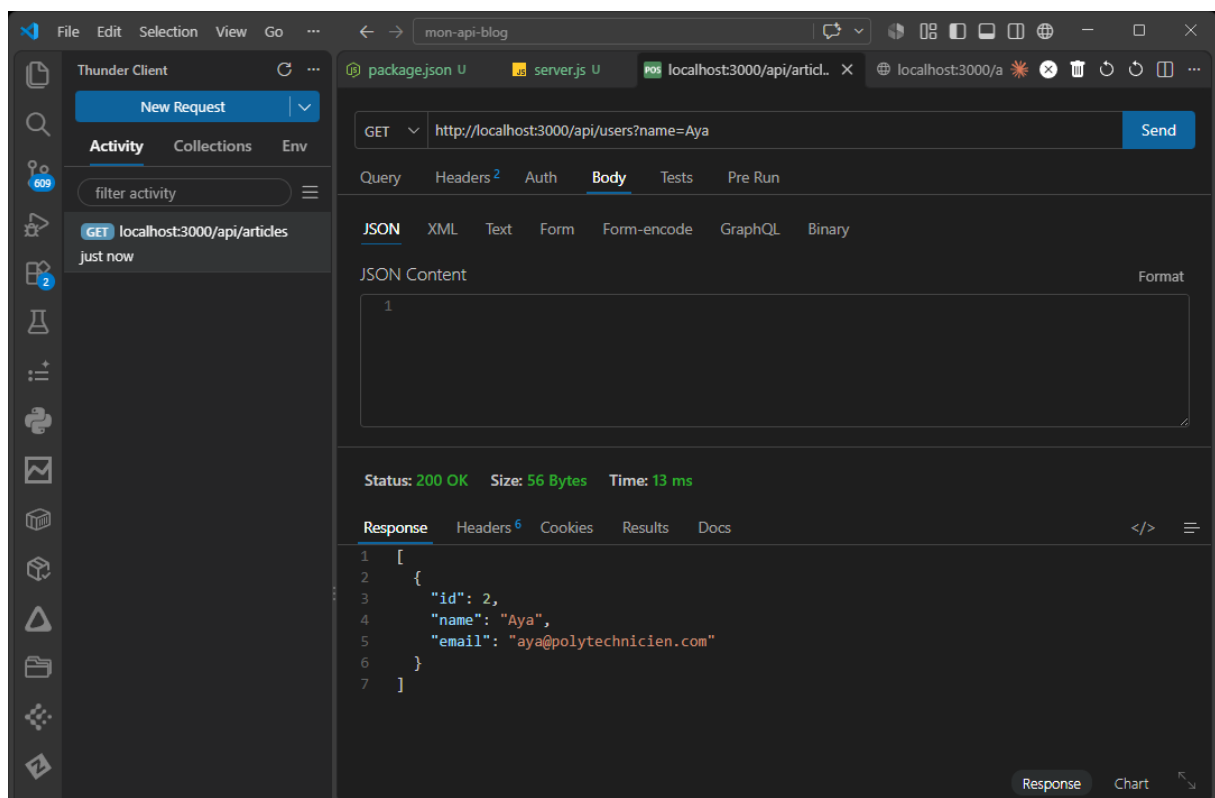
app.get("/api/users", (req, res) => {
  const name = req.query.name;

  if (!name) {
    return res.json(users);
  }

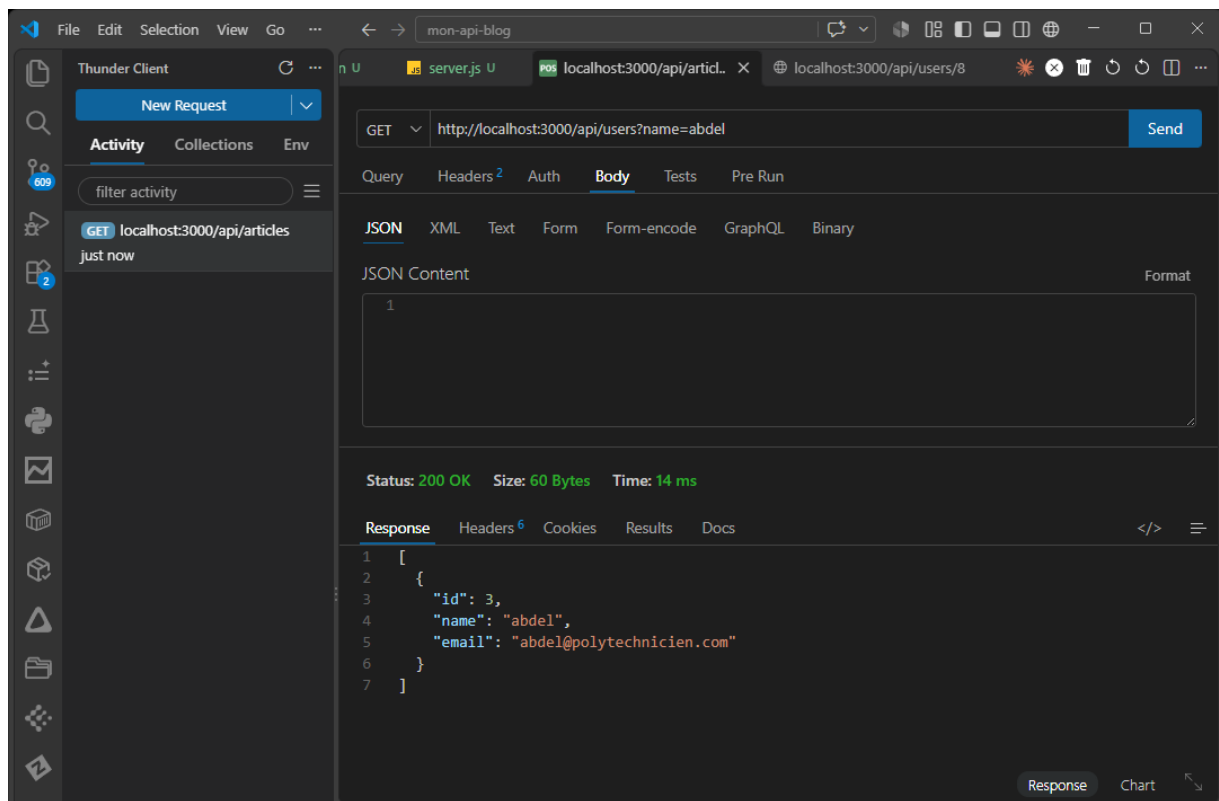
  const filteredUsers = users.filter(user =>
    user.name.toLowerCase() === name.toLowerCase()
  );

  res.json(filteredUsers);
});
```

En tester le filtre avec Thunder Client



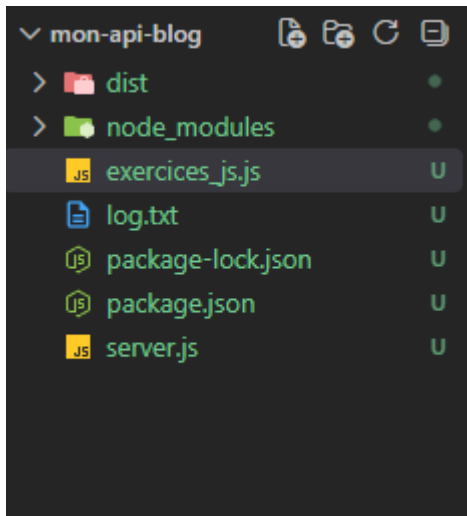
Un autre test avec Thunder Client



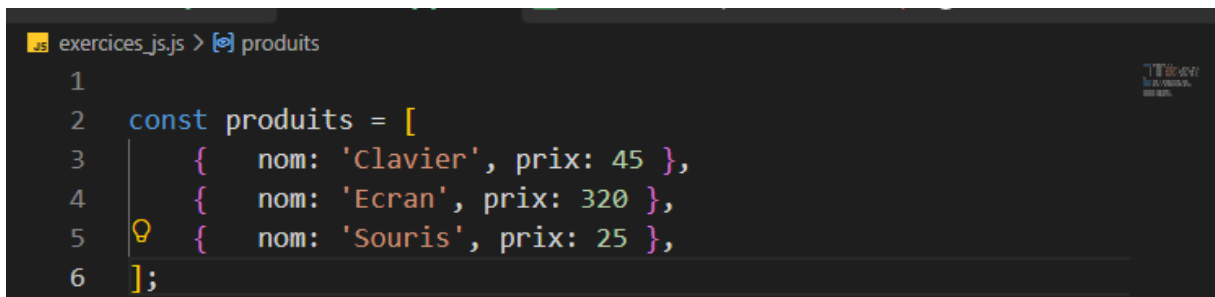
Exercice 2 :

Etap 1

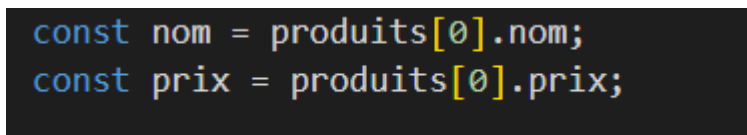
En creat un file avec nome « exercices_js.js »



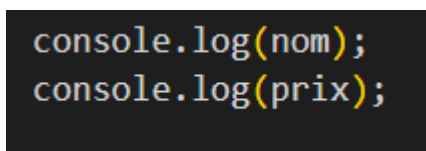
En enregtrr cette liste en nom produits



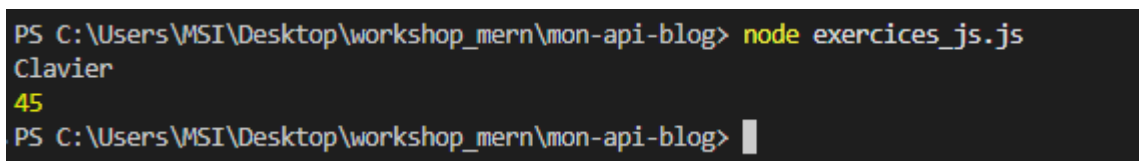
En faire un const avec « nom et prix » pour le premer produit



En utilise console.log() pour affiche le resultat



C'est le resultat



Etap 2

En utilise filtre par nom pour affiche le prix

```
exercices_js.js > ...
1
2 const produits = [
3   { nom: 'Clavier', prix: 45 },
4   { nom: 'Ecran', prix: 320 },
5   { nom: 'Souris', prix: 25 },
6 ];
7 const nom = produits[0].nom;
8 const prix = produits[0].prix;
9
10 console.log(nom);
11 console.log(prix);
12
13 const produit = produits.find(p => p.nom === "Souris");
14 console.log(produit.prix);
```

C'est le résultat

```
PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> node exercices_js.js
Clavier
45
25
```

J'utilise le symbole de commentaire pour annuler les résultat précédent

```
const produits = [
  { nom: 'Clavier', prix: 45 },
  { nom: 'Ecran', prix: 320 },
  { nom: 'Souris', prix: 25 },
];
const nom = produits[0].nom;
const prix = produits[0].prix;
// console.log(nom);
// console.log(prix);

const produit = produits.find(p => p.nom === "Souris");
console.log(produit.prix);
```

C'est la nouvelle résultat

```
PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> node exercices_js.js
25
PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> []
```

Etape 3

En utilise filtre avec le prix pour affiche les produit

```
exercices_js.js > ...
1
2  const produits = [
3    { nom: 'Clavier', prix: 45 },
4    { nom: 'Ecran', prix: 320 },
5    { nom: 'Souris', prix: 25 },
6  ];
7  const nom = produits[0].nom;
8  const prix = produits[0].prix;
9
10 // console.log(nom);
11 // console.log(prix);
12
13 const produit = produits.find(p => p.nom === "Souris");
14 // console.log(produit.prix);
15
16 const produitsFiltres = produits.filter(p => p.prix < 100);
17 console.log(produitsFiltres); |
```

C'est l'affichage de résultat

```
PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> node exercices_js.js  
[ { nom: 'Clavier', prix: 45 }, { nom: 'Souris', prix: 25 } ]  
❖ PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> 
```

Etape 4

En utilise filtre par nom et en ajoute le remise de 10%

```
// console.log(produitsFiltres);  
const ecran = produits.find(p => p.nom === "Ecran");  
  
const avecRemise = (prix) => {  
  return prix * ((100 - 10) / 100);  
};  
console.log(avecRemise(ecran.prix));
```

C'est l'affichage de résultat

```
PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> node exercices_js.js  
288  
❖ PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> 
```

C'est tous le code sans commentaire

```
exercices_js.js > ...
1
2  const produits = [
3      {    nom: 'Clavier', prix: 45 },
4      {    nom: 'Ecran',  prix: 320 },
5      {    nom: 'Souris', prix: 25 },
6  ];
7  const nom = produits[0].nom;
8  const prix = produits[0].prix;
9
10 console.log(nom);
11 console.log(prix);
12
13 const produit = produits.find(p => p.nom === "Souris");
14 console.log(produit.prix);
15
16 const produitsFiltres = produits.filter(p => p.prix < 100);
17 console.log(produitsFiltres);
18 const ecran = produits.find(p => p.nom === "Ecran");
19
20 const avecRemise = (prix) => {
21     return prix * ((100 - 10) / 100);
22 };
23 console.log(avecRemise(ecran.prix));
24
```

C'est tout affichage

```
288
● PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> node exercices_js.js
Clavier
45
25
[ { nom: 'Clavier', prix: 45 }, { nom: 'Souris', prix: 25 } ]
288
❖ PS C:\Users\MSI\Desktop\workshop_mern\mon-api-blog> 
```